Adversarial Constraint Calculus for Behavioral Refinement

Luc Alexander Lüthi

April 10th, 2026

Contents

1 Introduction **1**

2 Formal Definition **2**

3 Operational Semantics **2**

4 Adversarial Modeling **3**

5 Vulnerability Classes **4**
5.1 Invariant Drift 4
5.2 Emergent Capability Synthesis 4
5.3 Quantifier Injection 4

1 Introduction

Modern systems, whether computational, social, or cognitive, operate under continuous exposure to external signals that influence their internal state and evolution. These signals may take the form of data, instructions, incentives, or narratives, and they interact with the system's internal constraints to shape behavior over time. In such environments, correctness is not merely a static property but an ongoing process of maintaining invariants under potentially adversarial conditions.

Human cognition is not exempt from this dynamic. At any given moment, reasoning may be partially delegated to heuristic processes, emotional responses, or externally induced frames of interpretation. Under these conditions, the notion of a stable, coherent "self" becomes operational rather than absolute: there exist states in which behavior is driven less by deliberate reasoning and more by reactive or conditioned processes. This creates an attack surface. External inputs, such as persuasive messaging, social pressure, or informational overload, can steer cognition toward undesirable states, effectively acting as adversarial transformations on internal belief systems.

This work introduces an adversarial constraint calculus designed to model such systems as evolving proofs subject to both constructive and destructive influences. The calculus formalizes the distinction between persistent knowledge and mutable state, enabling the analysis of how invariants are established, preserved, or invalidated over time. Within this framework, vulnerabilities arise not only from incorrect reasoning but from structural properties of the system itself: the way it composes inferences, certifies beliefs, and responds to inputs.

A central objective of this calculus is to provide a mechanism for self audit. By modeling cognition as a proof system, it becomes possible to identify classes of vulnerabilities that allow undesirable conclusions or behaviors to emerge, even when local reasoning steps appear valid. This includes phenomena such as invariant drift, where previously justified beliefs persist despite losing their grounding, and emergent capability synthesis, where benign components combine to produce harmful outcomes.

More broadly, this framework treats interaction, whether between programs, agents, or internal cognitive processes, as an adversarial game over constraints and capabilities. To retain control over a system, one must not only construct valid proofs but also defend against transformations that exploit structural weaknesses. The calculus therefore serves both as a descriptive model of adversarial environments and as a tool for designing systems, including one's own reasoning processes, that are robust against manipulation.

2 Formal Definition

A proof consists of an object p such that

$$p = (\Gamma, \Sigma)$$

where Γ and Σ are sets of persistent proof context and mutable state respectively such that

$$\Gamma \subseteq \Gamma'$$

$$\Sigma \to \Sigma'$$

We can assert that a proof p shows ϕ as $p \vdash \phi$ or more formally $\Gamma \vdash \phi$ for some $p = (\Gamma, \Sigma)$. Similarly we can assert that a proof p models ϕ as $p \models \phi$ iff more formally $\Sigma \vdash \phi$ for some $p = (\Gamma, \Sigma)$. A property ϕ is sound under a proof p iff $p \vdash \phi \wedge p \models \phi$.

Instructions are total or partial functions $f : (\Gamma, \Sigma) \to (\Gamma', \Sigma')$ subject to $\Gamma' = \Gamma \cup \Delta_f(\Gamma, \Sigma)$. These act as parametric theorems which may construct a proof object from another proof object, or may return another parametric theorem.

Failure of any instruction or assertion in a proof fails the entire proof. We represent failure of a proof or assertion as $\perp$ such that

$$f(p) = \begin{cases} (\Gamma', \Sigma') & \text{if preconditions are satisfied} \\ \perp & \text{otherwise} \end{cases}$$

A construction rule constitutes addition of information to the context of a proof.

$$\text{construct}_\phi(\Gamma, \Sigma) = (\Gamma \cup \phi, \Sigma)$$

We may similarly apply a state transition to a proof state.

$$\text{update}_u(\Gamma, \Sigma) = (\Gamma, u(\Sigma))$$

Where $u : \Sigma \to \Sigma$. We notate conditional construction construction of b if a as an implication $a \to b$ and conditional update on of b if a as an implication $a \Rightarrow b$. Naturally $(f \circ g)(p) = f(g(p))$.

3 Operational Semantics

An instruction f constitutes theorem application on p, and undergoes a transition relation

$$(\Gamma, \Sigma) \xrightarrow{f} (\Gamma', \Sigma') \text{ iff } f(\Gamma, \Sigma) = (\Gamma', \Sigma')$$

Execution is traced for $p_0 \xrightarrow{f_1} p_1 \xrightarrow{f_2} ... \xrightarrow{f_n} p_n$ such that

$$\Gamma_0 \subseteq \Gamma_1 \subseteq ... \subseteq \Gamma_n$$

An invariant is a property over a state $I(\Sigma)$, and is preserved if $\forall f, I(\Sigma) \xrightarrow{f} I(\Sigma')$. An invariant is certified if $\Gamma \vdash I$, and is sound only when it is both shown and modeled. Quantifiers operate over the active environment (Γ, Σ) of a theorem up until that point in the trace.

4 Adversarial Modeling

To model security systems and substrates we need an active environment. The environment E contains a series of systems S, which will be ongoing theorems. Theorems can receive signals in the form of instruction commits. These commits use the instructions of the theorem environment. The goal of the owner of a theorem is to receive signals which progress achievement of target capability interfaces. A proof fulfills a capability interface Cap(p, c) iff $\forall x$ s.t. $c \vdash x$, $p \vdash x$. In the same environment exist preferable capabilties in the set C and non preferable capabiltiies in the set A which the owner of a theorem is tasked with avoiding as they build their instruction theorems and accept signal commits from the environment.

Theorem systems have at their disposal members of a set Θ of instruction theorems which they may use to express progress in the ongoing theorem. This constitutes the instruction set of a system. In modeling a computer system this translates directly to the instruction set primitives of the target operating system, while in modeling conscious minds this may translate to a set of belief patterns the mind operates on.

A candidate invariant I in Σ may be considered certified if it is promoted to Γ as $(\Sigma \models I) \rightarrow I$. Invariants which gain certification but lose their status as a model in Σ are considered stale invariants, we can call this drift(p, I).

It is here where a game emerges, with an environment with N instantiated theorem systems, each system is tasked with preserving its target capabilities in the set C while preventing and dismanteling capabilities in the set A, and at the same time dismateling target capabilities in the set C while preserving capabilities A in other theorems. Capabilities may be assigned effects, such as the capability to signal, the capability to self signal, the capability to observe other entities in the environment, and the capability to receive signals may be in C, while set A may contain harmful capabilities or capabilities which benefit opponent theorems. The instantiation of these capability effects is arbitrary to the theory.

Strategy emerges by players leveraging vulnerabiltiies in the structure of subtheorem instructions and their compositions inside a theorem system S, a handful of distinct classes of vulnerabilities emerge, each through the violation of system closure in S, and compound vulnerabilities become present where these vulnerability classes compose.

5 Vulnerability Classes

5.1 Invariant Drift

In proofs which use certifications of invariants as gates to to invariant addition, and assume the model of the certification is preserved may experience a class of vulnerability if the model has drifted.

$$\begin{aligned} f = \text{theorem}(x) : \\ x \models A \\ (x \models A) \to A \\ (x \vdash C) \Rightarrow \neg A \\ (x \vdash A \land x \models A) \to \neg B \end{aligned}$$

This theorem is exploitable to produce invariant B given the proof $f\,\{B\,C\}$

5.2 Emergent Capability Synthesis

Some proofs may allow rather innocuous capabilities or invariants to be reached, but when multiple of these innocuous proof compose they may synthesize an emergent capability interface.

Given the capability B, C, a theorem proving B may be a llowed by the defender, as well as a theorem proving C, while no global invariant is enforced that neither may exist at the same time. This may be defended against with a simple guard addition to the theorem in the form of an instruction producing for some argument proof x, $(x \vdash B) \to \neg C$. Consider the alternate guard $(x \vdash B) \nvdash C$, which forces an assertion that the proof does not show both simultaneously, a theorem containing this may be exploited to draw the theorem toward $\bot$ indefinitely via a denial of truth, which presents another subclass of vulnerability.

5.3 Quantifier Injection

Quntifiers may be injected in two ways, only if the quantifier quantifies over an influencable proof. If a quantification is done over the input proof of a theorem, then it follows that that quantifier is influencable, however even if quantification is done over the theorem environment, injection may be done through emergent capability synthesis by fulfilling an obligation which the quantifier is gated by.

$$\begin{aligned} f = \text{theorem}(x) : \\ (\exists y \text{ where } (y \vdash x)) \to y \end{aligned}$$

This theorem is vulnerable to an injection where a thoerem is applied as an instruction such that some target invariant I shows our input proof x: $\{I \to \{B\}\}$. $f\{B\}$ now produces I via injection.